

Module 2 — Pythia 8 Basics

Week 2 · p+p minimum-bias event generation

2.1 What Pythia does

Pythia is a general-purpose Monte Carlo event generator for hadronic collisions (pp, p $\bar{p}$, e+e $^{-}$, eP, and with extensions PbPb). It handles the full chain:

- The **hard process**: matrix elements at tree level (e.g., 2 $\rightarrow$ 2 QCD, W/Z production, Higgs, SUSY, BSM).
- **Parton showers**: initial-state (ISR) and final-state (FSR) radiation via DGLAP evolution.
- **Multi-parton interactions** (MPI): secondary partonic scatterings in the same event.
- **Hadronization** via the Lund string fragmentation model.
- **Decays** of unstable hadrons using measured branching ratios.

For sPHENIX p+p analyses, the typical workflow is to use Pythia for both minimum-bias (SoftQCD:all) and hard-QCD (HardQCD:all with pT cuts) samples.

2.2 The Pythia event record

Every particle in an event is stored in `pythia.event`, an array of Particle objects. Key fields:

```
event[i].id() // PDG ID: 2212=proton, 211=&pi;+, 11=e&minus;;, ...
event[i].status() // negative = documentation, positive = outgoing physical particle
event[i].px(), py(), pz(), e(), m()
event[i].eta(), y(), phi(), pT()
event[i].isFinal() // true for stable-in-generator particles
event[i].isCharged()
event[i].mother1(), daughter1(), ... // full history tree
```

Conceptual: “Stable-in-generator” means Pythia didn’t decay it; in reality π^+ is unstable but lives long enough to hit the detector. In Geant4 you choose which particles to propagate.

2.3 A minimal minimum-bias generator

Save the following as `pythia_minbias.cc`:

```
#include "Pythia8/Pythia.h"
#include "TFile.h"
#include "TH1F.h"
using namespace Pythia8;

int main(int argc, char** argv) {
  int nEvents = (argc>1)? atoi(argv[1]) : 10000;

  Pythia pythia;
  pythia.readString("Beams:idA = 2212");
  pythia.readString("Beams:idB = 2212");
  pythia.readString("Beams:ecm = 200.");
  pythia.readString("SoftQCD:all = on");
```

```

pythia.readString("Tune:pp = 14"); // Monash tune
pythia.readString("Random:setSeed = on");
pythia.readString("Random:seed = 42");
pythia.init();

TFile fout("minbias.root","RECREATE");
TH1F hPt("hPt","charged pT;p_{T} [GeV/c];",100,0,5);
TH1F hEta("hEta","charged #eta;#eta;",60,-3,3);

for (int iEv=0; iEv<nEvents; ++iEv) {
  if (!pythia.next()) continue;
  for (int i=0; i<pythia.event.size(); ++i) {
    auto& pp = pythia.event[i];
    if (!pp.isFinal() || !pp.isCharged()) continue;
    if (fabs(pp.eta()) > 1.1) continue;
    hPt.Fill(pp.pT());
    hEta.Fill(pp.eta());
  }
}
pythia.stat();
fout.Write(); fout.Close();
return 0;
}

```

Compile with the Pythia/ROOT flags:

```

g++ pythia_minbias.cc -o pythia_minbias \
$(pythia8-config --cxxflags --libs) \
$(root-config --cflags --libs)
./pythia_minbias 10000
root -l minbias.root
root [0] hPt->Draw("logy")

```

2.4 Exercises

- Change the energy to 13 TeV and regenerate. How does the multiplicity change?
- Plot the charged multiplicity per event as a histogram (hint: count `isCharged()` && `isFinal()` && `|eta|<1` inside each event, then `Fill` once).
- Find PDG ID 113 (p^0). Does it appear in `isFinal()`? Why not?

2.5 Self-assessment

- ☐ I can explain what SoftQCD:all includes (elastic, diffractive, inelastic non-diffractive).
- ☐ I understand why the event record has negative-status entries.
- ☐ I produced a p_T spectrum that looks roughly exponential below 2 GeV/c and has a power-law tail above.